Chapter six

Transport Layer

Transport Layer

- It is responsible for process-to-process delivery of the entire message.
- A process is an application program running on a host.
- The Transport layer segments and reassembles data into a data stream.

Transport Layer

- Services located in the Transport layer both segment and reassemble data from/to upper-layer applications and unite it onto the same data stream.
- Ensuring that all the packets of a message sent by a source node arrive at the intended destination.

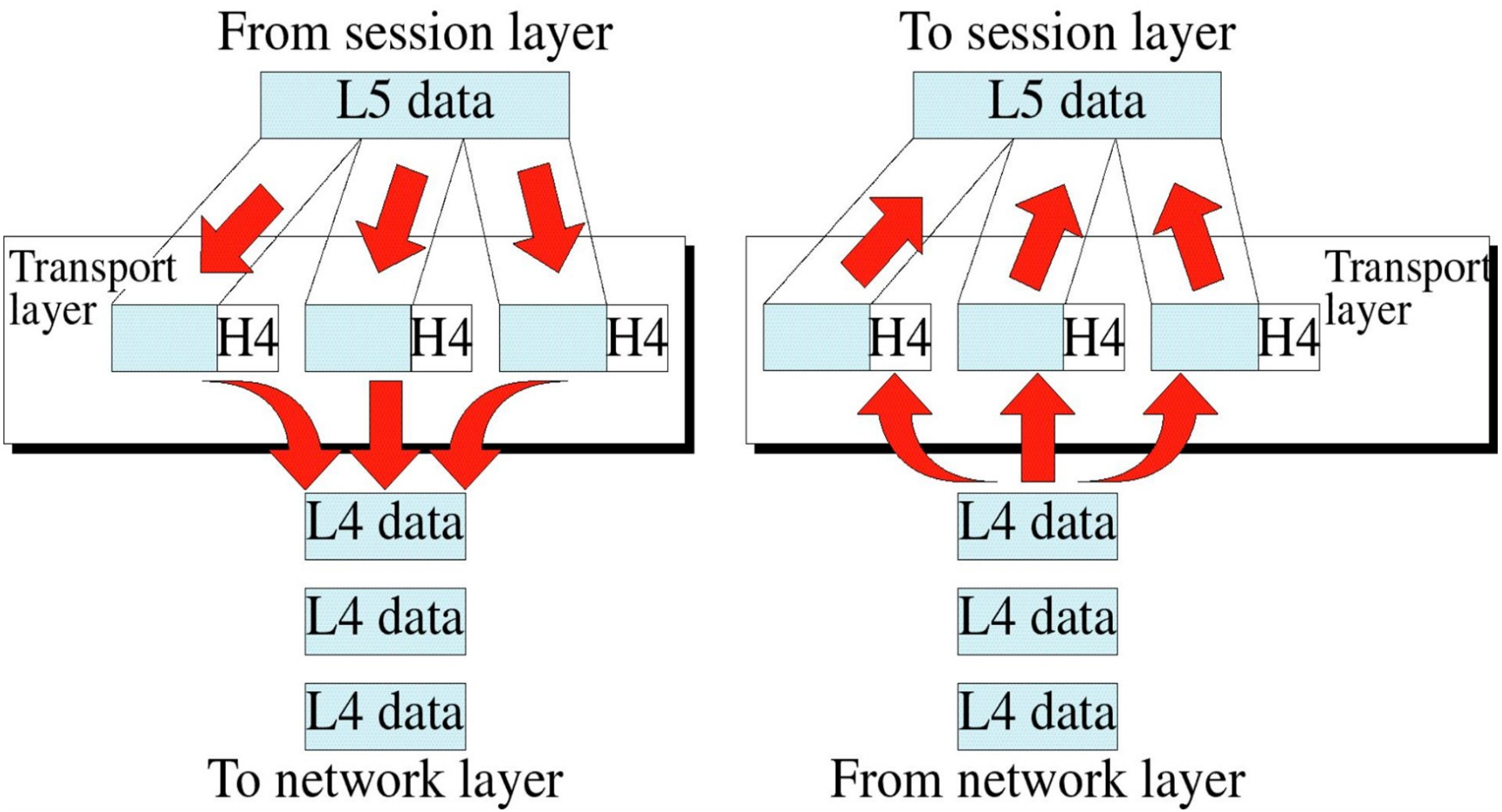
Cont..

- It ensures that the whole message arrives intact and in order, overseeing both **error control** and **flow control** at the source-to-destination level.
- The basic function of the transport layer is to accept data from above it, split it up into smaller units, pass these to the network layer, and ensure that the pieces all arrive correctly at the other end.

The transport layer is:

- responsible for logical communications between applications running on different hosts.
- The link between the application layer and the lower layers that are responsible for network transmission.

Cont..


From session layer
L5 data
Transport layer
H4
H4
H4
L4 data
L4 data
L4 data
To network layer
To session layer
L5 data
Transport layer
H4
H4
H4
L4 data
L4 data
L4 data
From network layer

Functions of Transport Layer

- **Segmentation** of message into packet & reassembly of packets into message.
- **Port addressing:** Computers run several processes.
- Transport layer header include a port address with each process.
- **Flow Control:** Flow control facility prevents the source form sending data packets faster than the destination can handle.
- **Error control**: Transport layer ensures that the entire message arrives at the receiving Transport layer without error.

Segmentation

- A message is divided into transmittable segments, with each segment containing a sequence number.
- These numbers enable the transport layer to reassemble the message correctly upon arriving at the destination and to identify and replace packets that were lost in transmission.

Addressing

- Computers often run several programs at the same time.
- For this reason, source-to-destination delivery means delivery not only from one computer to the next but also from a specific process (running program) on one computer to a specific process (running program) on the other.
- The transport layer header must therefore include a type of address called a *service-point address* (or port address).
- The network layer gets each packet to the correct computer; the transport layer gets the entire message to the correct process on that computer.

Multiplexing and DE-multiplexing

Multiplexing

- At the sender site, there may be several processes that need to send packets.
- However, there is only one transport layer protocol at any time.
- This is a many-to-one relationship and it requires multiplexing.
- The protocol accepts messages from different processes, differentiated by their assigned port numbers.
- After adding the header, the transport layer passes the packet to the network layer.

Cont..

De-multiplexing

- At the receiver site, the relationship is one-to-many and requires de-multiplexing.
- The transport layer receives datagrams from the network layer.
- After error checking and dropping of the header, the transport layer delivers each message to the appropriate process based on the port number.

Cont..

- **For example**
- Suppose you are sitting in front of your computer, and
- you are downloading Web pages while running one FTP session and two Telnet sessions.
- You therefore have four network application processes running—two Telnet processes, one FTP process, and one HTTP process.

Cont..

- When the transport layer in your computer receives data from the network layer below,
- it needs to direct the received data to one of these four processes.
- Let's now examine how this is done.
  - First recall that a process (as part of a network application) can have one or more sockets that are doors through which data passes from the network to the process and through which data passes from the process to the network.

Cont..

- Thus, as shown in Fig. 1, the transport layer in the receiving host does not actually deliver data directly to a process, but instead to an intermediary socket.
  - The combination of the source IP address and source port number, or the destination IP address and destination port number is known as a **socket**.
- Because at any given time there can be more than one socket in the receiving host, each socket has a unique identifier.

Cont..

- The format of the identifier depends on whether the socket is a UDP or a TCP socket.
- Now let's consider how a receiving host directs an incoming transport layer segment to the appropriate socket.
  - Each transport-layer segment has a set of fields in the segment for this purpose.

Cont..

- At the receiving end, the transport layer examines these fields to identify the receiving socket and then directs the segment to that socket.
  - This job of delivering the data in a transport-layer segment to the correct socket is called **demultiplexing**.
  - The job of gathering data chunks at the source host from different sockets, encapsulating each data chunk with header information (that will later be used in demultiplexing) to create segments, and passing the segments to the network layer is called **multiplexing**.

Cont..

- Note that the transport layer in the middle host in Fig. 1 must demultiplex segments arriving from the network layer below to either process P or P above; this is done by directing the arriving segment's data to the corresponding process's socket.
- The transport layer in the middle host must also gather outgoing data from these sockets, form transport layer segments, and pass these segments down to the network layer.

Cont..

The source port number serves as part of a "return address".

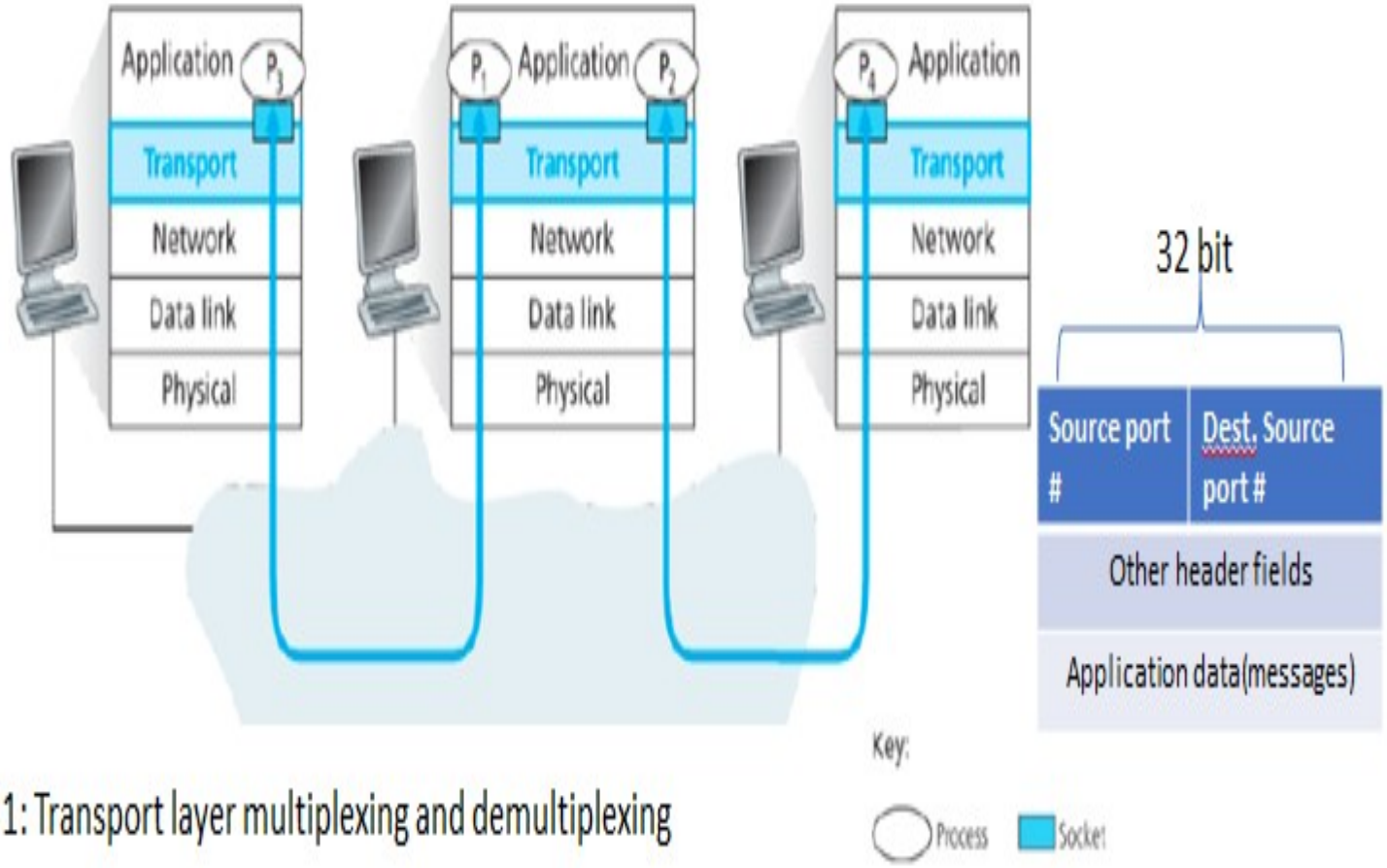


Fig. 1: Transport layer multiplexing and demultiplexing

Connectionless/Connection-Oriented Service

- A transport layer protocol can either be connectionless or connection-oriented.

Connectionless Service

- In a connectionless service, the packets are sent from one party to another with no need for connection establishment or connection release.
- The packets are not numbered; they may be delayed or lost or may arrive out of sequence.

Connectionless/Connection-Oriented Service

- In this type of transmission the receiving devices does not sends an acknowledgement back to the source.
- It is a faster transmission method.
- A connectionless transport layer treats each segment as an independent packet and delivers them to the transport layer at the destination machine.

Cont..

Connection-Oriented Service

- In a connection-oriented service, a connection is first established between the sender and the receiver.
- In this type of transmission the receiving devices sends an acknowledgement back to the source after a packet or group of packet are received.

Cont..

- It is slower transmission method.
- For reliable transport to occur, a device that wants to transmit must first establish a connection-oriented communication session with a remote device.
- This communication is known as a call setup or a three-way handshake.

Cont..

- The first “connection agreement” segment is a request for synchronization (SYN).
- The next segments acknowledge (ACK) the request and establish connection parameters (the rules between hosts).

Cont..

- These segments request that the receiver's sequencing is synchronized here as well so that a bidirectional connection can be formed.
- The final segment is also an acknowledgment, which notifies the destination host that the connection agreement has been accepted and that the actual connection has been established. Data transfer can now begin.

Unreliable / Reliable Service

- The transport layer service can be reliable or unreliable. If the application layer program needs reliability, we use a reliable transport layer protocol by implementing flow and error control at the transport layer.
- This means a slower and more complex service.

Unreliable / Reliable Service

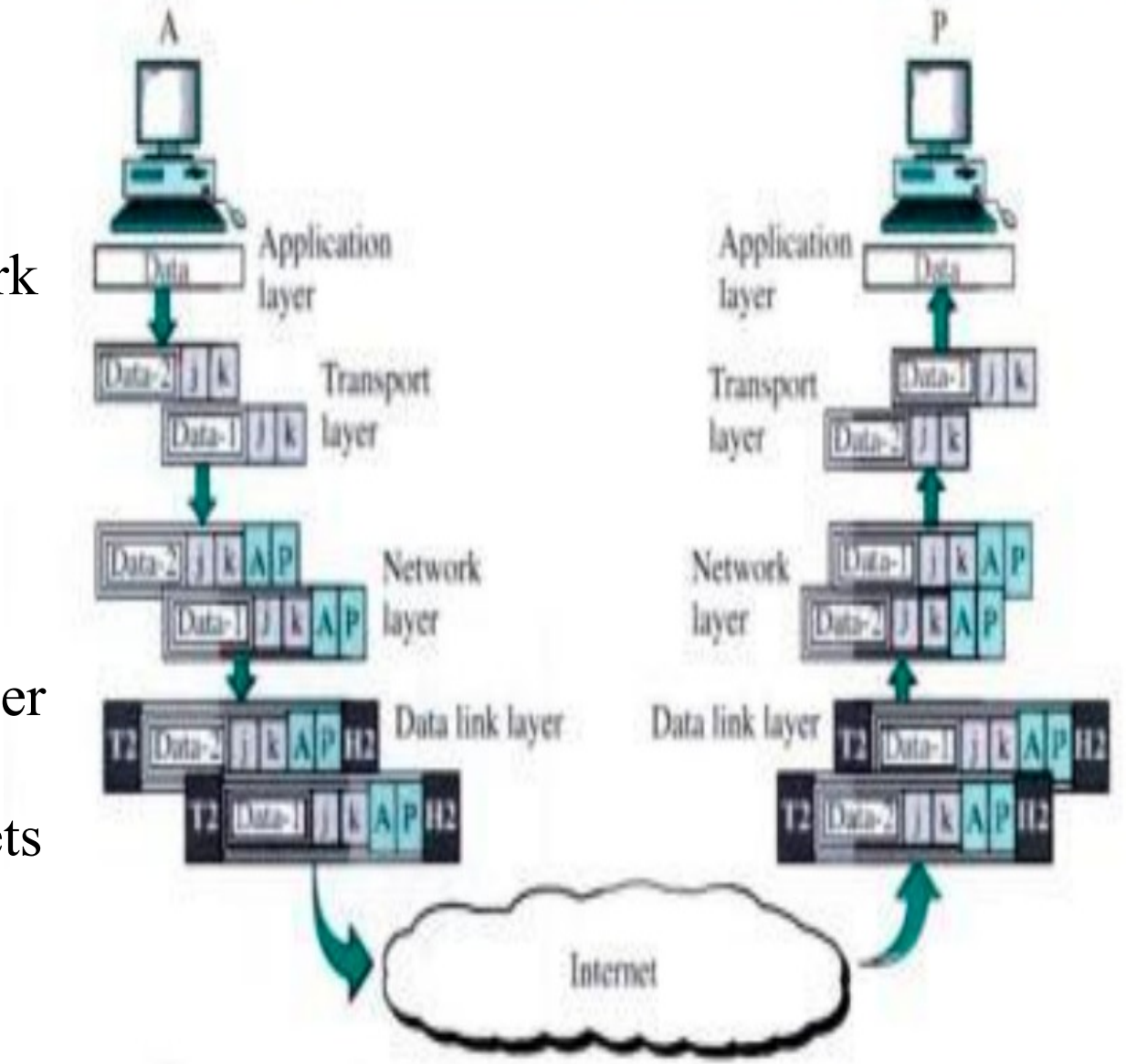
- On the other hand, if the application program does not need reliability because it uses its own flow and error control mechanism or it needs fast service or the nature of the service does not demand flow and error control (real-time applications), then an unreliable protocol can be used.
- In the Internet, there are two common different transport layer protocols.

Cont..

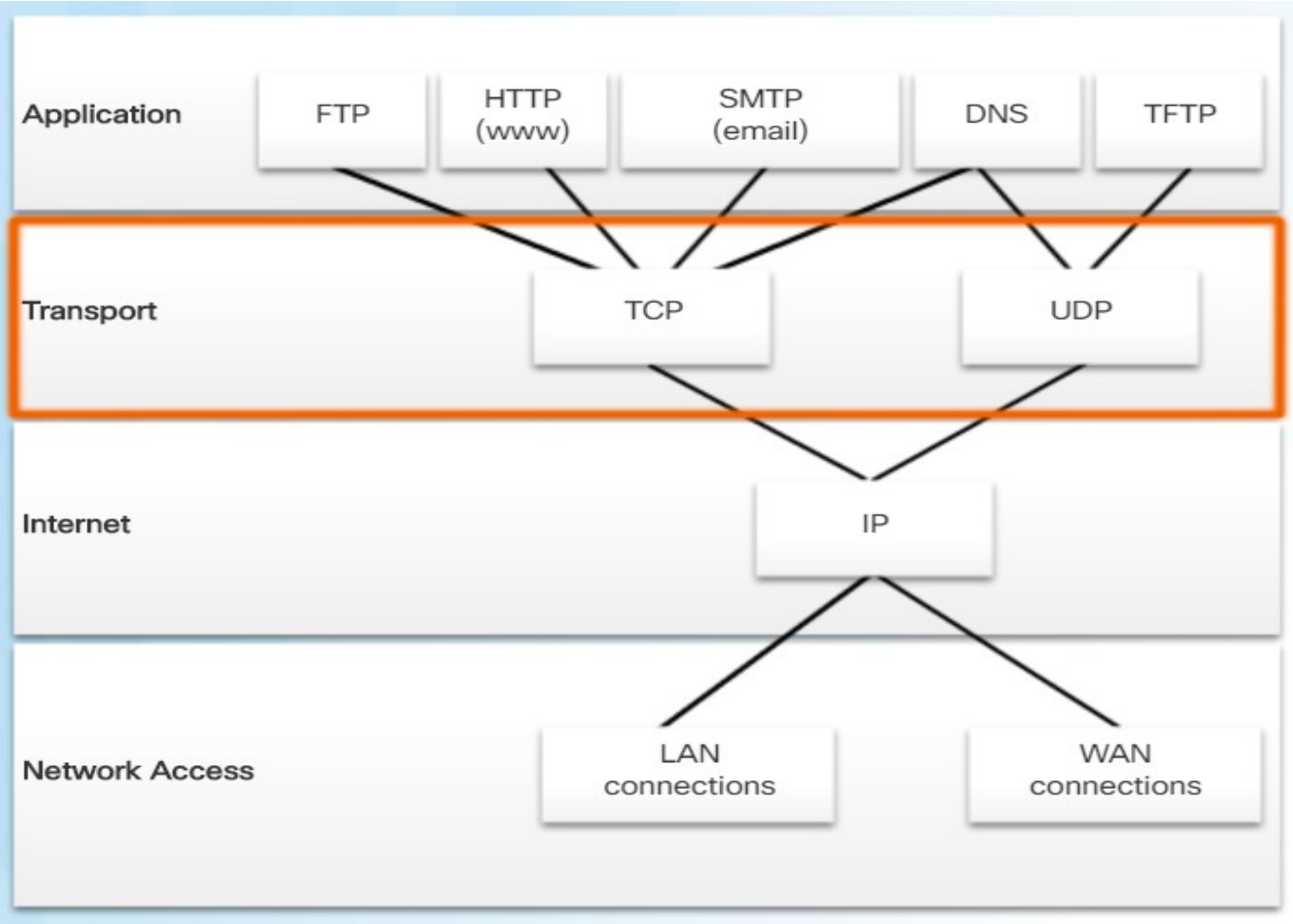
- UDP is connectionless and unreliable;
- TCP is a connection-oriented and reliable.
- Reliability at the data link layer is between two nodes; we need reliability between two ends.
- Because the network layer in the Internet is unreliable (best-effort delivery), reliability need to be implemented at the transport layer.

Example

- Since the data size is larger than the network layer can handle; the data is split into two packets, each packets retaining the port addresses.
- Then in Network Layer added a network address to each packets



Transport Layer Protocols



Cont..

- The Internet's network-layer protocol has a name—IP, for Internet Protocol.
- IP provides logical communication between hosts.
- The IP service model is a best-effort delivery service.
  - This means that IP makes its "best effort" to deliver segments between communicating hosts, but it makes no guarantees.
- In particular, it does not guarantee
  - segment delivery,
  - orderly delivery of segments, and
  - the integrity of the data in the segments.
- For these reasons, IP is said to be an unreliable service.

Cont..

- The most fundamental responsibility of UDP and TCP is to extend IP's delivery service between two end systems to a delivery service between two processes running on the end systems.
- Extending host-to-host delivery to process-to-process delivery is called transport-layer multiplexing and demultiplexing.
- UDP and TCP also provide integrity checking by including error
- These are two minimal transport-layer services
  - process-to-process data delivery and
  - error checking

User Datagram Protocol / UDP

- The User Datagram Protocol (UDP) is called a connectionless, unreliable transport protocol.
- It does not add anything to the services of IP except to provide process-to-process communication instead of host-to-host communication.

•In particular, like IP, UDP is an unreliable service—it does not guarantee that data sent by one process will arrive intact (or at all!) to the destination process.

UDP features include the following:

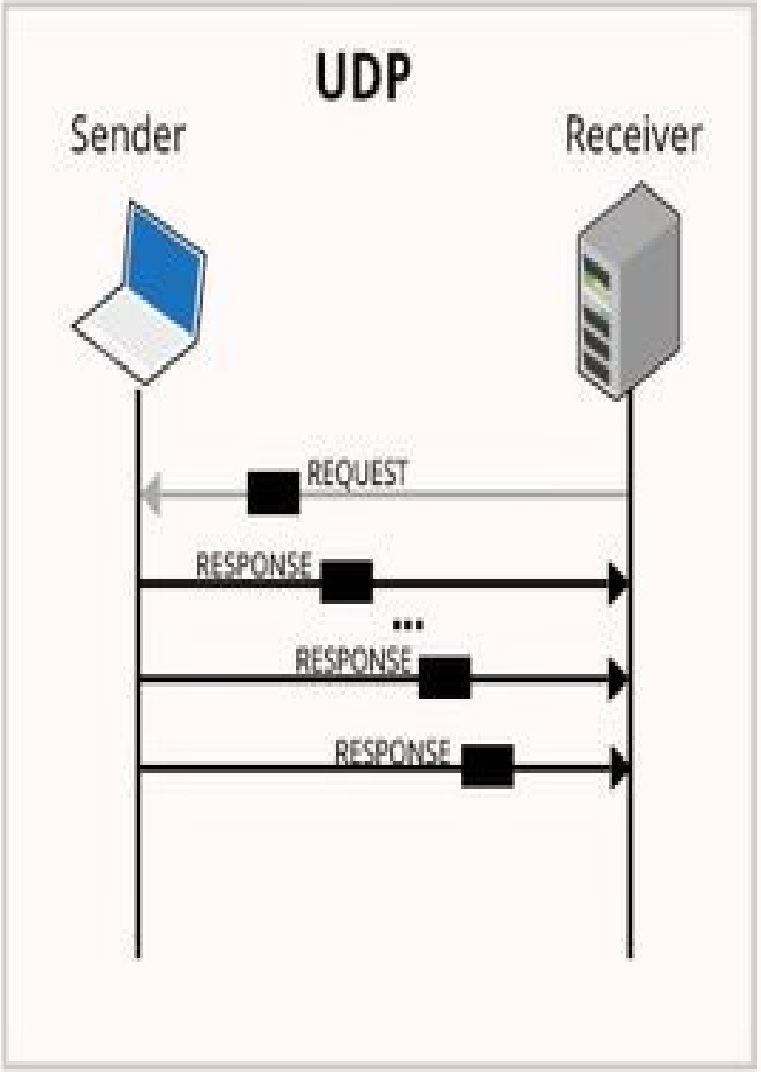
- Data is reconstructed in the order that it is received.
- Any segments that are lost are not resent.
- There is no session establishment.
- The sending is not informed about resource availability.

Cont..

- It also performs very limited error checking.
- If UDP is so powerless, why would a process want to use it? With the disadvantages come some advantages.
- UDP is a very simple protocol using a minimum of overhead.
- If a process wants to send a small message and does not care much about reliability, it can use UDP.
- Sending a small message by using UDP takes much less interaction between the sender and receiver than using TCP.

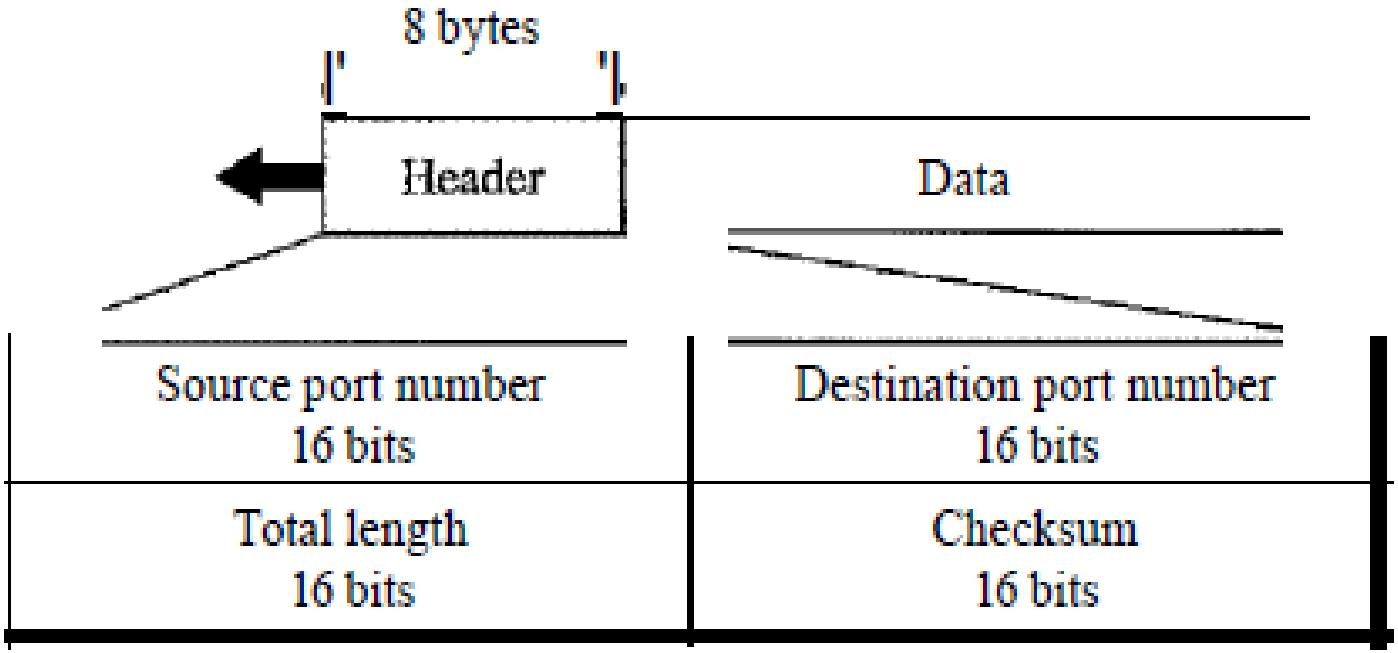
Cont..

- UDP does not track sequence numbers the way TCP does.
- UDP has no way to reorder the datagrams into their transmission order.
- UDP simply reassembles the data in the order that it was received and forwards it to the application.



Cont..

- UDP packets, called user datagrams, have a fixed-size header of 8 bytes.



Cont..

- The fields are as follows:

Source port number - This is the port number used by the process running on the source host.

Destination port number - This is the port number used by the process running on the destination host.

Length - This is a 16-bit field that defines the total length of the user datagram, header plus data.

Checksum - This field is used to detect errors over the entire user datagram (header plus data).

Applications that use UDP

- **Live video and multimedia applications** - These applications can tolerate some data loss but require little or no delay. Examples include VoIP and live streaming video.
- **Simple request and reply applications** - Applications with simple transactions where a host sends a request and may or may not receive a reply. Examples include DNS and DHCP.
- **Applications that handle reliability themselves** - Unidirectional communications where flow control, error detection, acknowledgments, and error recovery is not required, or can be handled by the application. Examples include SNMP and TFTP.

Transmission Control Protocol / TCP

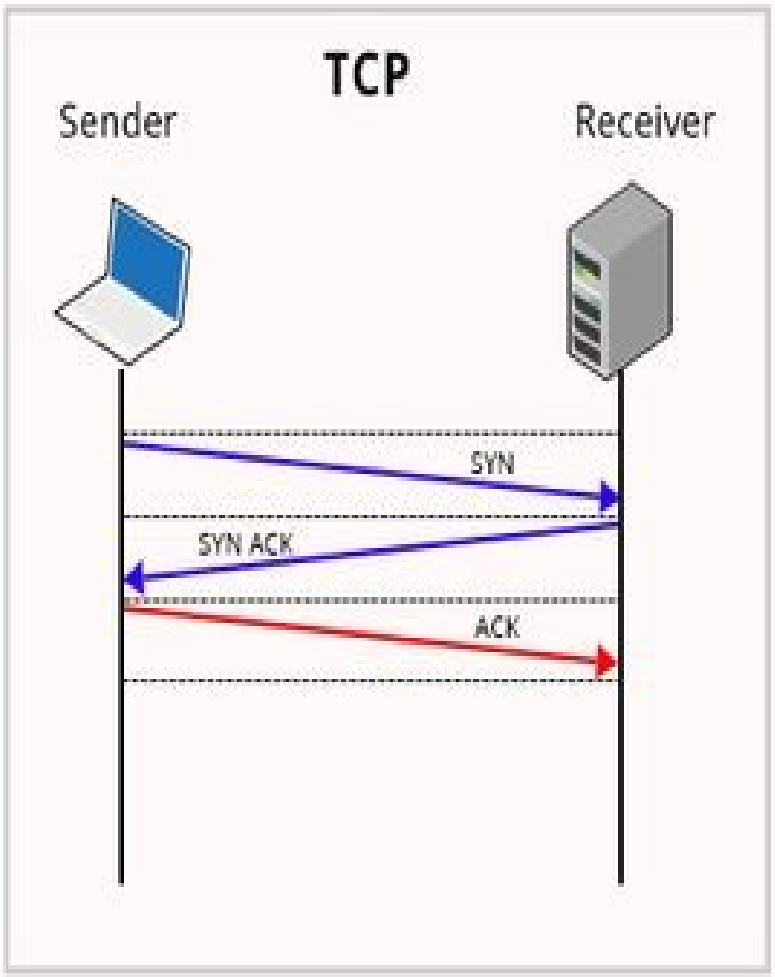
- TCP, like UDP, is a process-to-process (program-to-program) protocol.
- TCP, therefore, like UDP, uses port numbers.
- Unlike UDP, TCP is a connection-oriented protocol; it creates a virtual connection between two TCPs to send data.
- In addition, TCP uses flow and error control mechanisms at the transport level.
- In brief, TCP is called a connection-oriented, reliable transport protocol.
- It adds connection-oriented and reliability features to the services of IP.

Cont..

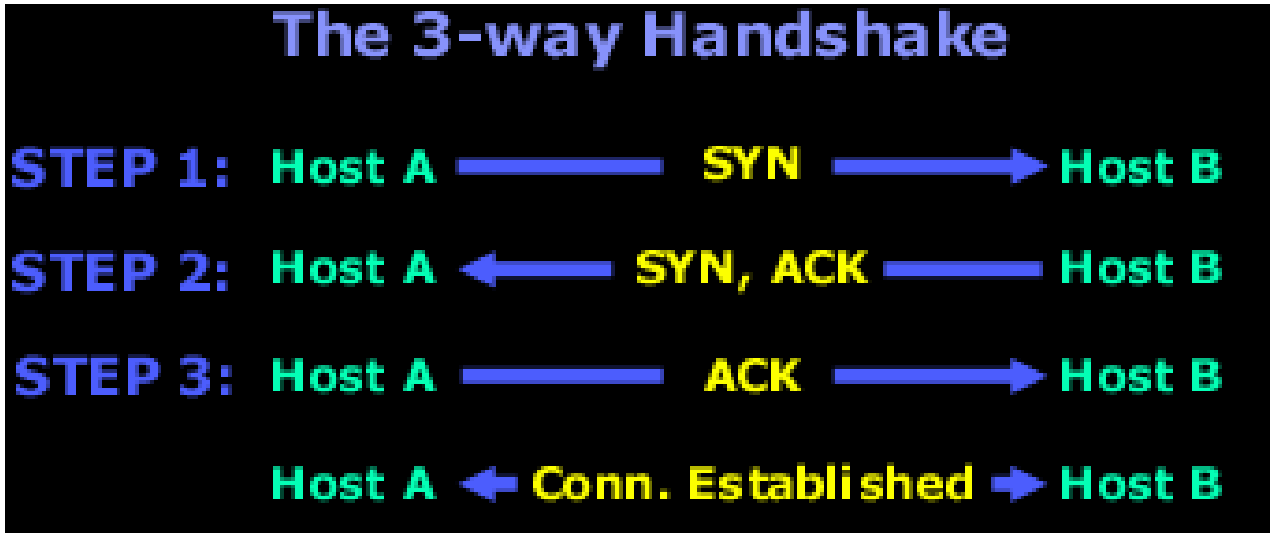
•TCP provides reliability and flow control.

TCP basic operations:

- Number and track data segments transmitted to a specific host from a specific application
- Acknowledge received data
- Retransmit any unacknowledged data after a certain amount of time
- Sequence data that might arrive in wrong order
- Send data at an efficient rate that is acceptable by the receiver
- TCP provides multiple services. They are listed as follows.



The following diagram explains the basic function of the 3-way handshake:



STEP 1: Host A sends a packet to Host B. This packet has the "SYN" bit enabled and when Host B receives it and reads the packet, it sees the "SYN" bit which has a value of "1" (in binary, this means ON) so it knows that Host A is trying to synchronise with it.
STEP 2: Host B then sends a packet back to Host A and within this packet, the "SYN and ACK" bits are enabled (value =1). The SYN that Host B sends means 'I want to synchronise with you' and the ACK means 'I acknowlege your previous SYN request'.
STEP 3: So... after all that, Host A sends another packet to Host B and has the "ACK" bit set to 1, which tells Host B 'Yes, I acknowlege your previous request'.
And after all that, the connection is established (virtual circuit) and the data transfer begins, and should end without any errors!

Flow control

- TCP, unlike UDP, provides flow control.
- The receiver of the data controls the amount of data that are to be sent by the sender.
- This is done to prevent the receiver from being overwhelmed with data.
- The numbering system allows TCP to use a byte-oriented flow control.

Error control

- To provide reliable service, TCP implements an error control mechanism.
- The transport layer is responsible for error control.
- The sending transport layer makes sure that the entire message arrives at the receiving transport layer without error (damage, loss, or duplication).
- Error correction is usually achieved through retransmission.

Congestion control

- TCP, unlike UDP, takes into account congestion in the network.
- The amount of data sent by a sender is not only controlled by the receiver (flow control), but is also controlled by the level of congestion in the network.

TCP Variants

- TCP is mostly used Internet Protocol.
- The main feature of TCP is that it can handle the congestion.
- When the packet sending rate is increased than the receiving rate then congestion arises.
- TCP provides the reliable and connection oriented network.
- To deals with congestion, TCP provides different variants for example:- TCP Reno, TCP SACK ..

Cont..

TCP Reno

- It was introduced in 1990 by Van Jacobson.
- In TCP Reno when three duplicate packets are received, then it is the sign of congestion.
- If congestion occurs, then TCP Reno retransmits the packets and enters a new mechanism that is fast recovery.

Cont..

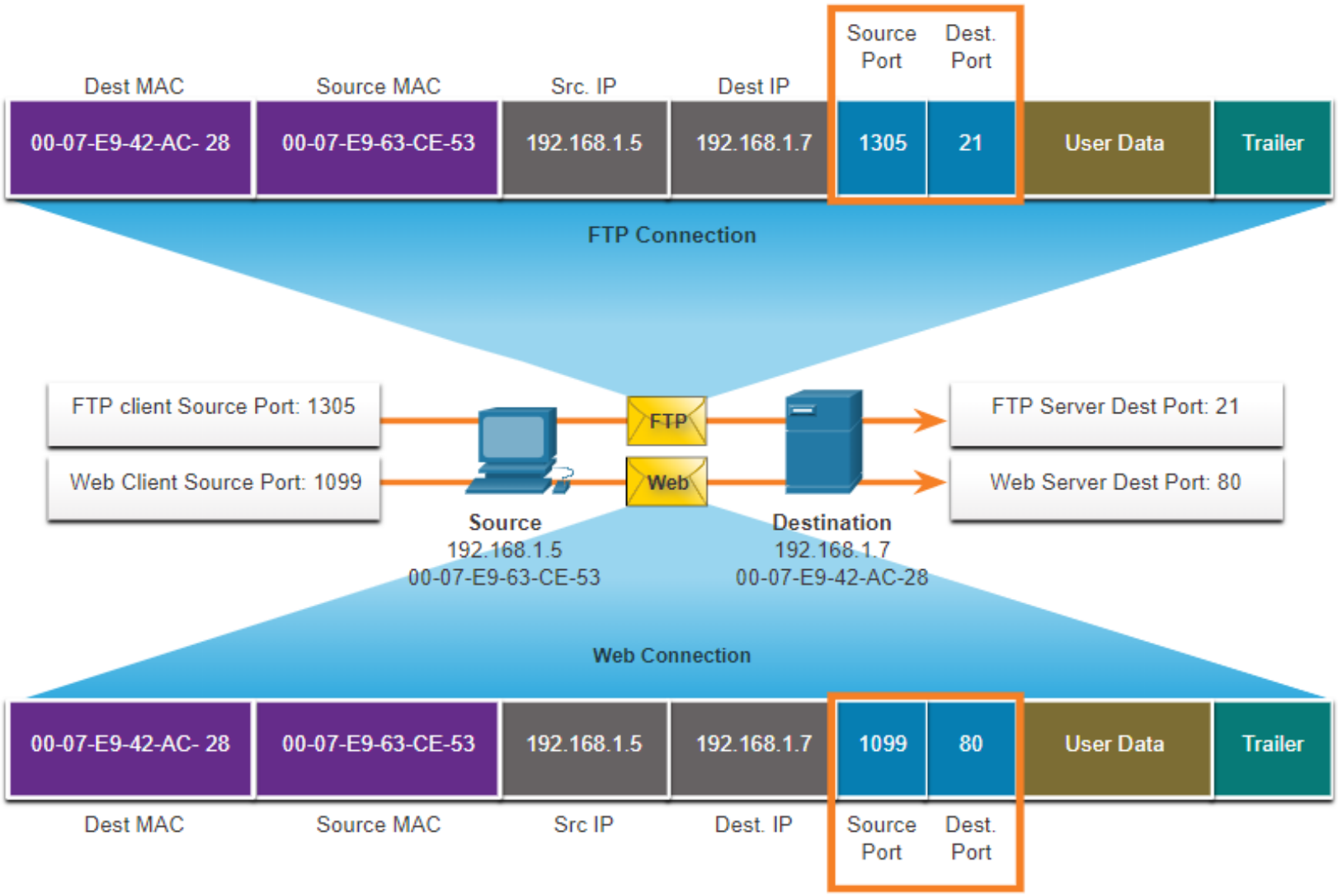
TCP SACK

- TCP SACK or selective acknowledgement requires that packets should acknowledge selectively.
- It is an option enabling a receiver to tell the sender the range of non-contiguous packets received.
- Without SACK, the receiver can only tell the sender about sequentially received packets.
- The sender uses this information to retransmit selectively only the lost packets.
- It uses a variable pipe to store outstanding data in the network, which is absent in TCP Reno and New Reno.

Port numbers and Socket Pairs

- The source and destination ports are placed within the segment.
- The segments are then encapsulated within an IP packet.
- The combination of the source IP address and source port number, or the destination IP address and destination port number is known as a socket.
- Sockets enable multiple processes, running on a client, to distinguish themselves from each other, and multiple connections to a server process to be distinguished from each other.

Cont..


Source Port
Dest. Port
Dest MAC
Source MAC
Src. IP
Dest IP
00-07-E9-42-AC- 28
00-07-E9-63-CE-53
192.168.1.5
192.168.1.7
1305
21
User Data
Trailer
FTP Connection
FTP client Source Port: 1305
Web Client Source Port: 1099
FTP
Web
FTP Server Dest Port: 21
Web Server Dest Port: 80
Source
192.168.1.5
00-07-E9-63-CE-53
Destination
192.168.1.7
00-07-E9-42-AC-28
Web Connection
00-07-E9-42-AC- 28
00-07-E9-63-CE-53
192.168.1.5
192.168.1.7
1099
80
User Data
Trailer
Dest MAC
Source MAC
Src IP
Dest. IP
Source Port
Dest. Port

Cont..

Port Group	Number Range	Description
Well-known Ports	**0 to 1,023**	•These port numbers are reserved for common or popular services and applications such as web browsers, email clients, and remote access clients. •Defined well-known ports for common server applications enables clients to easily identify the associated service required.
Registered Ports	**1,024 to 49,151**	•These port numbers are assigned by IANA to a requesting entity to use with specific processes or applications. •These processes are primarily individual applications that a user has chosen to install, rather than common applications that would receive a well-known port number. •For example, Cisco has registered port 1812 for its RADIUS server authentication process.
Private and/ or **Dynamic Ports**	**49,152 to 65,535**	•These ports are also known as *ephemeral ports*. •The client's OS usually assign port numbers dynamically when a connection to a service is initiated. •The dynamic port is then used to identify the client application during communication.

Port Number	Protocol type	Application
20	TCP	File Transfer Protocol (FTP) - Data
21	TCP	File Transfer Protocol (FTP) - Control
22	TCP	Secure Shell (SSH)
23	TCP	Telnet
25	TCP	Simple Mail Transfer Protocol (SMTP)
53	UDP, TCP	Domain Name Service (DNS)
67	UDP	Dynamic Host Configuration Protocol (DHCP) - Server
68	UDP	Dynamic Host Configuration Protocol - Client
69	UDP	Trivial File Transfer Protocol (TFTP)
80	TCP	Hypertext Transfer Protocol (HTTP)
110	TCP	Post Office Protocol version 3 (POP3)
143	TCP	Internet Message Access Protocol (IMAP)
161	UDP	Simple Network Management Protocol (SNMP)
443	TCP	Hypertext Transfer Protocol Secure (HTTPS)

End of chapter 6